Adaptive LLM Routing Framework - Comprehensive Technical Document

1. Project Overview

Project Name & Description

The Adaptive LLM Routing Framework is a production-grade orchestration engine designed to dynamically route user prompts to the most efficient Large Language Model (from a pool of 15 candidates) based on predicted quality and cost metrics.

Problem Statement & Motivation

Monolithic reliance on frontier models (like GPT-4) is expensive, slow, and computationally wasteful for simple tasks. While open-weight models are cheaper and faster, they lack the reliability required for complex reasoning tasks. Static routing policies cannot adapt to the nuance of individual queries, leading to either bloated costs or sub-optimal responses.

Goals & Expected Outcomes

- Develop a cost-aware routing policy engine capable of achieving ~95% of GPT-4's baseline quality.
- Drastically reduce average time-to-first-token (TTFT) and API cost by intelligently leveraging smaller models for simpler prompts.
- Establish a self-correcting evaluation loop to ensure high reliability.

2. Evolution of Approach & The "Jugaad" (Workaround)

Building this project required pivoting from our initial assumptions due to real-world infrastructure constraints.

Initial Approach (Rejected)

Initially, we planned to route execution literally to the 15 models our TensorFlow

evaluators were trained on (e.g., Llama-2-70b, Falcon-40b, Vicuna-33b) using Hugging Face's free serverless inference API.

- *Why it failed:* Hugging Face retired free serverless endpoints for massive models. Models like 70B require 80GB-140GB of VRAM, making them impossible to run locally or for free. Furthermore, models like Llama-2 are "gated" and require manual license approvals, breaking automated API calls.

The "Jugaad" (Decoupled Proxy Execution)

To bypass this API roadblock without spending hundreds of dollars on paid cloud GPU clusters, we implemented a "Decoupled Routing Simulation."

We separated the *decision intelligence* from the *infrastructure execution*.

1. **Intelligence**: Agent 1 uses our 15 trained local `.weights.h5` TensorFlow models to score and predict performance for the original legacy models.
2. **Execution**: Once Agent 1 selects a legacy model (e.g., Vicuna-33b), our executor maps it to an equivalent modern model hosted on **Groq's ultra-fast, free LPU tier**.

The Proxy Map:

- 7B–13B Tier models → Routed to `openai/gpt-oss-20b` on Groq.
- 30B–40B Tier models → Routed to `qwen/qwen3.8-27b` on Groq.
- 65B–70B Tier models → Routed to `openai/gpt-oss-120b` on Groq.

This "jugaad" allowed us to prove our TensorFlow routing logic mathematically, while executing text generation at 800+ tokens/second for free.

3. Technical Architecture & Data Flow

Architecture Diagram

```
None
graph TD
    A[User Query + Constraints] --> B[Generate 384-dim Query Embedding]
    B --> C{Semantic Cache in Qdrant <br> Similarity > 0.90?}
    C -- Yes --> D[Return Cached Response]
    C -- No --> E[Agent 1: Model Selection]
```

```
E --> F[Run Embedding through 15 TF Models]
F --> G[Predict Scores: Helpfulness, Tokens, Overall]
G --> H[Apply Routing Strategy <br> e.g., Best Quality / Cost Optimized]
H --> I[Execute Query via API <br> Groq Proxy / OpenAI]
I --> J{Agent 2: Response Evaluation}
J -- Acceptable --> K[Store in Semantic Cache & Return to User]
J -- Failed --> L[Add to Eliminated Models List & Retry Agent 1]
```

Data Flow Explanation

3. **Ingestion & Caching**: A user submits a prompt. We generate a dense vector embedding using `all-MiniLM-L6-v2`. We query Qdrant (using `query_points`); if a match > 0.90 similarity is found, we immediately return the cached text, skipping LLM execution entirely.
4. **Evaluation Array**: The prompt embedding is fed directly into 15 pre-loaded TensorFlow models simultaneously. We execute them as direct callables (e.g., `model(tensor)`) rather than using `model.predict()` to avoid expensive graph retracing bottlenecks.
5. **Execution**: The executor takes the winning model, maps it to the actual API endpoint (via our Groq proxy strategy or OpenAI for GPTs), and streams the generation.
6. **Validation Loop**: Agent 2 performs a sanity check. If the LLM generated garbage or an API failed, it flags the model, adds it to an "eliminated" list, and forces Agent 1 to pick the *second-best* model for a retry.

4. Technology Stack

- **FastAPI**: Highly concurrent, low-latency API backend. Chosen for its native `async` support which prevents blocking during external API calls.
- **Qdrant**: Local vector database for semantic caching. Chosen for sub-millisecond retrieval speeds.
- **TensorFlow / Keras**: Used for the 15 multi-head neural networks predicting model utility.

- **SentenceTransformers**: Specifically `all-MiniLM-L6-v2`, for extremely fast, robust text embeddings.
- **External Providers**: Groq (for LPU-accelerated proxy models), OpenAI, and HuggingFace Inference Providers.

5. Cost & Efficiency Tracking

To prove the framework's viability for the thesis, the database logs two distinct metrics for every query:
1. Simulated Thesis Cost: What it *would* have cost using the 15 original models (calculated mathematically).
2. Actual API Cost: The literal billing cost executed on Groq/OpenAI.
This allows us to visually graph the financial efficiency of dynamic routing versus a static "always use GPT-4" baseline.

6. Thesis Defense Q&A Prep

Anticipated questions from faculty and technical explanations.
Q1: Why train 15 separate TensorFlow models instead of one massive classifier?
A1: Modular architecture. By isolating models, we can add a new LLM to our candidate pool (or remove a deprecated one) without having to retrain the entire massive neural network. It keeps the system plug-and-play.
Q2: How did you fix the severe TensorFlow performance lag during the routing phase?
A2: Initially, using `model.predict()` in a loop triggered expensive tf.function retracing because it's built for batching. We bypassed this by converting the embedding to a tensor and invoking the model directly as a callable (`preds = model_instance(prompt_tensor, training=False)`).
Q3: How do you handle hallucinations or API failures from open-source models?
A3: We implemented a self-correcting loop (Agent 2). If an API times out or the response is invalid, Agent 2 returns a `REMEDIATED_FALLBACK` status, adds the failed model to the `eliminated_models` list, and automatically retries the prompt with the next best-scored model.

Q4: Why use a 0.90 threshold for the Semantic Cache?

A4: With `all-MiniLM-L6-v2`, >0.95 is too strict (missing paraphrases), and <0.85 is too lenient (causing false positives where unrelated questions trigger the cache). 0.90 is the industry-standard "sweet spot" for balancing cost savings with cache accuracy.

Q5: What happens if an API provider changes their library, like Qdrant recently did?

A5: We experienced this during development when `qdrant-client` deprecated the `search()` method. We updated our microservice wrapper to use `query_points()` without breaking the rest of the application, proving the resilience of our decoupled architecture.